

# **LAPORAN JOBSHEET 14**

## **TREE**

**Nama : Rachmah Nur Chotimah**

**NIM : 254107060052**

**Kelas : SIB 1A**

**No.Absen : 20**

## 14.2 Kegiatan Praktikum 1

### Implementasi Binary Search Tree menggunakan Linked List (100 Menit)

#### 14.2.2 Pertanyaan Percobaan

1. Mengapa dalam binary search tree proses pencarian data bisa lebih efektif dilakukan dibanding binary tree biasa?

Jawab :

Binary Search Tree (BST) memiliki aturan bahwa nilai yang lebih kecil dari suatu node ditempatkan di cabang kiri dan nilai yang lebih besar di cabang kanan, sehingga saat proses pencarian kita dapat langsung mengabaikan setengah bagian pohon yang tidak mungkin berisi data yang dicari (jika nilai lebih kecil menuju kiri, jika lebih besar menuju kanan), yang membuat kompleksitas pencarian rata-rata menjadi jauh lebih efisien dibandingkan Binary Tree biasa yang memerlukan pencarian satu per satu hingga seluruh node.

2. Untuk apakah di class **Node**, kegunaan dari atribut **left** dan **right**?

Jawab :

Atribut **left** dan **right** bertipe objek **Node20** yang berfungsi sebagai pointer atau referensi alamat memori ke node anak,

- **left** menunjuk ke anak sebelah kiri yang memiliki nilai IPK lebih kecil
- **right** menunjuk ke anak sebelah kanan yang memiliki nilai IPK lebih besar

sehingga kedua atribut ini memungkinkan terbentuknya struktur pohon karena menghubungkan setiap node dengan node lainnya.

3. a. Untuk apakah kegunaan dari atribut **root** di dalam class **BinaryTree**?  
b. Ketika objek tree pertama kali dibuat, apakah nilai dari **root**?

Jawab :

- a. Kegunaan root adalah sebagai **pintu masuk utama** atau fondasi awal dari seluruh struktur pohon (akar). Karena *pohon* diakses dari atas ke bawah, variabel root bertugas menyimpan alamat elemen paling atas. Semua operasi seperti penambahan (add), pencarian (find), dan penghapusan (delete) wajib dimulai dengan membaca posisi root terlebih dahulu.
- b. Nilainya adalah **null**. Hal ini menandakan bahwa pohon baru saja dibuat dan masih dalam kondisi kosong (belum memiliki data/node sama sekali).

4. Ketika tree masih kosong, dan akan ditambahkan sebuah node baru, proses apa yang akan terjadi?

Jawab :

Sistem akan mengecek kondisi lewat method `isEmpty()`. Karena `root == null` (kosong), maka node baru yang dibuat (`newNode`) akan langsung ditunjuk dan diangkat menjadi **root utama pohon**. Alur tersebut terdapat pada kode program berikut:

```
if (isEmpty()) {  
    root = newNode;
```

5. Perhatikan method **add()**, di dalamnya terdapat baris program seperti di bawah ini. Jelaskan secara detil untuk apa baris program tersebut?

```
parent = current;  
if (mahasiswa.ipk < current.mahasiswa.ipk) {  
    current = current.left;  
    if (current == null) {  
        parent.left = newNode;  
        return;  
    }  
} else {  
    current = current.right;  
    if (current == null) {  
        parent.right = newNode;  
        return;  
    }  
}
```

Jawab :

Baris kode tersebut digunakan untuk mencari posisi kosong yang sesuai untuk menempatkan data mahasiswa baru pada **Binary Search Tree (BST)** berdasarkan nilai IPK, dengan cara menyimpan node saat ini sebagai **parent**, kemudian membandingkan IPK data baru dengan IPK node yang sedang diperiksa; jika lebih kecil maka bergerak ke cabang kiri (**current = current.left**), sedangkan jika lebih besar atau sama maka bergerak ke cabang kanan (**current = current.right**), dan ketika ditemukan posisi kosong (**current == null**), node baru akan dihubungkan ke **parent** sebagai anak kiri atau anak kanan sesuai hasil perbandingan, lalu proses penambahan data selesai.

6. Jelaskan langkah-langkah pada method **delete()** saat menghapus sebuah node yang memiliki dua anak. Bagaimana method **getSuccessor()** membantu dalam proses ini?

Jawab :

Saat menghapus node yang memiliki dua anak, proses tidak dapat dilakukan secara langsung karena dapat merusak struktur pohon, sehingga program terlebih dahulu mencari successor (penerus), yaitu node dengan nilai terkecil pada sub-pohon kanan menggunakan method **getSuccessor()**. Method ini melepaskan successor dari posisi lamanya sambil menjaga hubungan dengan node lain yang masih terhubung, kemudian successor dipindahkan untuk menggantikan posisi node yang dihapus. Setelah itu, anak kiri dan anak kanan dari node lama disambungkan ke successor

sehingga seluruh cabang pohon tetap terhubung. Dengan cara ini, `getSuccessor()` membantu memilih pengganti yang paling tepat sehingga aturan Binary Search Tree tetap terjaga, yaitu semua nilai di cabang kiri lebih kecil dan semua nilai di cabang kanan lebih besar daripada node induknya.

## 14.3 Kegiatan Praktikum 2

### Implementasi Binary Tree dengan Array (45 Menit)

#### 14.3.2 Pertanyaan Percobaan

1. Apakah kegunaan dari atribut `data` dan `idxLast` yang ada di class **BinaryTreeArray**?

Jawab :

- **dataMahasiswa[]** berfungsi sebagai wadah utama berbentuk array statis untuk menyimpan objek-objek mahasiswa yang membentuk struktur **binary tree**,
- **idxLast** digunakan untuk menandai indeks terakhir yang berisi data valid dalam array sehingga dapat menjadi batas proses perulangan atau rekursi dan mencegah program mengakses indeks kosong yang dapat menyebabkan error.

2. Apakah kegunaan dari method **populateData()**?

Jawab :

Method **populateData()** digunakan untuk memasukkan atau menyalin sekumpulan data awal sekaligus dari main class ke dalam array **dataMahasiswa** (`dataMhs`) pada kelas **BinaryTreeArray (BTArray)**, sehingga proses inisialisasi struktur pohon dapat dilakukan secara cepat dan praktis tanpa perlu menambahkan data satu per satu menggunakan method **insert()**.

3. Apakah kegunaan dari method **traverseInOrder()**?

Jawab :

Method **traverseInOrder()** digunakan untuk menelusuri dan menampilkan seluruh data mahasiswa yang tersimpan dalam pohon biner menggunakan pola **In-Order** (*Kiri → Root → Kanan*) secara rekursif, sehingga pada struktur **Binary Search Tree (BST)** data akan ditampilkan secara otomatis dalam urutan menaik (*ascending*) berdasarkan nilai kuncinya.

4. Jika suatu node binary tree disimpan dalam array indeks 2, maka di indeks berapakah posisi left child dan right child masing-masing?

Jawab :

Berdasarkan representasi **binary tree** menggunakan array, posisi anak kiri (*left child*) dihitung dengan rumus  $2 \times \text{idxStart} + 1$  dan posisi anak kanan (*right*

*child*) dihitung dengan rumus  $2 \times \text{idxStart} + 2$ . Oleh karena itu, jika node induk berada pada indeks 2, maka:

- Anak kiri (left) =  $2 \times 2 + 1 = 5$
- Anak kanan (right) =  $2 \times 2 + 2 = 6$

anak kiri berada pada indeks 5 dan anak kanan berada pada indeks 6.

5. Apa kegunaan statement `int idxLast = 6` pada praktikum 2 percobaan nomor 4?

Jawab :

Statement `int idxLast = 6` digunakan untuk menandai bahwa data aktif yang membentuk struktur **binary tree** dalam array hanya berada pada indeks **0 hingga 6**, kemudian nilai tersebut dikirim ke method `populateData()` agar fungsi rekursif `traverseInOrder()` mengetahui batas akhir data yang valid sehingga proses penelusuran dan pencetakan berhenti pada indeks ke-6 serta tidak mengakses indeks berikutnya yang masih bernilai **null**.

6. Mengapa indeks  $2 \times \text{idxStart} + 1$  dan  $2 \times \text{idxStart} + 2$  digunakan dalam pemanggilan rekursif, dan apa kaitannya dengan struktur pohon biner yang disusun dalam array?

Jawab :

Indeks  $2 \times \text{idxStart} + 1$  dan  $2 \times \text{idxStart} + 2$  digunakan karena merupakan rumus standar untuk merepresentasikan hubungan antara **parent**, **left child**, dan **right child** pada **binary tree** yang disimpan dalam bentuk array. Dengan rumus tersebut, program dapat menelusuri anak kiri melalui indeks  $2 \times \text{idxStart} + 1$  dan anak kanan melalui indeks  $2 \times \text{idxStart} + 2$ , sehingga meskipun tidak menggunakan pointer **left** dan **right** seperti pada implementasi berbasis node, struktur hierarki serta proses traversal binary tree tetap dapat direpresentasikan dan diakses secara akurat dalam memori.

## 14.4 Tugas Praktikum

1. Buat method di dalam class `BinaryTree00` yang akan menambahkan node dengan cara rekursif (`addRekursif()`).

```
//Tugas 1 : Menambahkan Node secara rekursif
public void addRekursif(Mhs20 mhs) {
    root = addRekursifHelper(root, mhs);
}

private Node20 addRekursifHelper(Node20 current, Mhs20 mhs) { // untuk menambahkan node secara rekursif dengan method bantuan/helper
    if (current == null) { // jika posisi kosong
        return new Node20(mhs);
    }
    if (mhs.ipk < current.mhs.ipk) { // jika ipk yang akan ditambahkan lebih kecil dari ipk pada node saat ini, maka akan ditambahkan ke subtree kiri
        current.left = addRekursifHelper(current.left, mhs);
    } else { // jika ipk yang akan ditambahkan lebih besar atau sama dengan ipk pada node saat ini, maka akan ditambahkan ke subtree kanan
        current.right = addRekursifHelper(current.right, mhs);
    }
    return current;
}
```

2. Buat method di dalam class `BinaryTree00` untuk menampilkan data mahasiswa dengan IPK paling kecil dan IPK yang paling besar (`cariMinIPK()` dan `cariMaxIPK()`) yang ada di dalam binary search tree.

```
// Tugas 2 : Mencari Data Mahasiswa dengan ipk terkecil
public void findMin() {
    if (isEmpty()) {
        System.out.println(x: "Binary Tree kosong");
        return;
    }
    Node20 current = root;
    while (current.left != null) { // untuk mencari node dengan ipk terkecil, kita akan terus menelusuri subtree kiri hingga mencapai node paling kiri
        current = current.left;
    }
    System.out.println(x: "Data mahasiswa dengan ipk terkecil: ");
    current.mhs.tampilkan();
}

// Tugas 2 : Mencari Data Mahasiswa dengan ipk terbesar
public void findMax() {
    if (isEmpty()) {
        System.out.println(x: "Binary Tree kosong");
        return;
    }
    Node20 current = root;
    while (current.right != null) { // untuk mencari node dengan ipk terbesar, kita akan terus menelusuri subtree kanan hingga mencapai node paling kanan
        current = current.right;
    }
    System.out.println(x: "Data mahasiswa dengan ipk terbesar: ");
    current.mhs.tampilkan();
}
}
```

3. Buat method dalam class BinaryTree00 untuk menampilkan data mahasiswa dengan IPK di atas suatu batas tertentu, misal di atas 3.50 (tampilMahasiswaIPKdiAtas(double ipkBatas)) yang ada di dalam binary search tree.

```
// Tugas 3 : Mencari Data Mahasiswa dengan ipk di atas Batas Tertentu
public void findAbove(double batas) {
    if (isEmpty()) {
        System.out.println(x: "Binary Tree kosong");
        return;
    }
    System.out.println("Data mahasiswa dengan ipk di atas " + batas + ": ");
    findAboveHelper(root, batas);
}

// method bantuan/helper untuk mencari data mahasiswa dengan ipk di atas batas tertentu secara rekursif
private void findAboveHelper(Node20 current, double batas) {
    if (current != null) {
        findAboveHelper(current.left, batas);
        if (current.mhs.ipk > batas) {
            current.mhs.tampilkan();
        }
        findAboveHelper(current.right, batas);
    }
}
}
```

Hasil dari penambahan tiga method diatas:

```
=====
                        PENGUJIAN TUGAS
=====

Data mahasiswa dengan ipk terkecil:
NIM : 244160185 Nama : Candra Kelas : C IPK : 3.21
Data mahasiswa dengan ipk terbesar:
NIM : 244160221 Nama : Badar Kelas : B IPK : 3.85
Data mahasiswa dengan ipk di atas 3.5:
NIM : 244160220 Nama : Dewi Kelas : B IPK : 3.54
NIM : 244160131 Nama : Devi Kelas : A IPK : 3.72
NIM : 244160221 Nama : Badar Kelas : B IPK : 3.85
```

4. Modifikasi class BinaryTreeArray00 di atas, dan tambahkan : • method add(Mahasiswa data) untuk memasukkan data ke dalam binary tree • method traversePreOrder()

```
//Tugas 4 : Method add() untuk menambahkan data mahasiswa ke dalam array
void add(Mhs20 mhs) {
    if (idxLast >= dataMhs.length - 1) { // mengecek apakah masih ada ruang untuk menambahkan data mahasiswa
        System.out.println("Array sudah penuh, tidak dapat menambahkan data mahasiswa.");
        return;
    }
    idxLast++;
    dataMhs[idxLast] = mhs;
    System.out.println("Data mahasiswa berhasil ditambahkan: " + mhs.nama + " berhasil ditambahkan ke index " + idxLast);
}

// Tugas 4 : Method traversePreOrder()
void traversePreOrder(int idxStart) {
    // POLA : ROOT -> Subtree Kiri -> Subtree Kanan
    if (idxStart <= idxLast) {
        if (dataMhs[idxStart] != null) {
            dataMhs[idxStart].tampilkan();
            traversePreOrder(2 * idxStart + 1); // rekursif ke subtree kiri
            traversePreOrder(2 * idxStart + 2); // rekursif ke subtree kanan
        }
    }
}
}
```

Hasil dari penambahan 2 method diatas :

```
=== PENGUJIAN METHOD ADD() ARRAY ===
Data mahasiswa berhasil ditambahkan: Ali berhasil ditambahkan ke index 0
Data mahasiswa berhasil ditambahkan: Candra berhasil ditambahkan ke index 1
Data mahasiswa berhasil ditambahkan: Badar berhasil ditambahkan ke index 2
Data mahasiswa berhasil ditambahkan: Dewi berhasil ditambahkan ke index 3
Data mahasiswa berhasil ditambahkan: Devi berhasil ditambahkan ke index 4
Data mahasiswa berhasil ditambahkan: Ehsan berhasil ditambahkan ke index 5
Data mahasiswa berhasil ditambahkan: Fizi berhasil ditambahkan ke index 6

=====
Inorder Traversal Mahasiswa (Kiri -> Root -> Kanan):
=====
NIM : 244160220 Nama : Dewi Kelas : B IPK : 3.35
NIM : 244160185 Nama : Candra Kelas : C IPK : 3.41
NIM : 244160131 Nama : Devi Kelas : A IPK : 3.48
NIM : 244160121 Nama : Ali Kelas : A IPK : 3.57
NIM : 244160205 Nama : Ehsan Kelas : D IPK : 3.61
NIM : 244160221 Nama : Badar Kelas : B IPK : 3.75
NIM : 244160170 Nama : Fizi Kelas : B IPK : 3.86

=====
PreOrder Traversal Mahasiswa (Root -> Kiri -> Kanan):
=====
NIM : 244160121 Nama : Ali Kelas : A IPK : 3.57
NIM : 244160185 Nama : Candra Kelas : C IPK : 3.41
NIM : 244160220 Nama : Dewi Kelas : B IPK : 3.35
NIM : 244160131 Nama : Devi Kelas : A IPK : 3.48
NIM : 244160221 Nama : Badar Kelas : B IPK : 3.75
NIM : 244160205 Nama : Ehsan Kelas : D IPK : 3.61
NIM : 244160170 Nama : Fizi Kelas : B IPK : 3.86
```

**Link GitHub :** <https://github.com/lovvovala/https---github.com-lovvovala-PASD/tree/main/Jobsheet14>